

Web Penetration Testing Assessment

Burp Suite Proxy History Analysis

Date: January 2, 2026

Analysis Type: Passive Traffic Analysis

Tool: Burp Suite Professional

Step 1: Scope Identification

Target URL (In-Scope)

Primary Target: <http://testphp.vulnweb.com>

Scope Definition:

- The target application is explicitly a TEST and demonstration site for Acunetix Web Vulnerability Scanner
- Site disclaimer states: "This website was constructed with common web programming errors so it is buggy"
- The application is intentionally vulnerable for security testing purposes

Out-of-Scope Domains:

- www.google.com (38 requests - user search activity)
 - www.gstatic.com (3 requests)
 - www.googletagmanager.com (2 requests)
 - play.google.com (2 requests)
 - Other Google infrastructure
-

Step 2: Traffic Analysis

Volume Analysis

- **Total HTTP Requests in Proxy History:** ~50 requests
- **In-Scope Requests (testphp.vulnweb.com):** ~30 requests
- **Out-of-Scope Requests:** ~20 requests (Google infrastructure)

Technology Stack Fingerprinting

Web Server:

- **nginx/1.19.0** (Released 2020)
 -  **FINDING:** Outdated version with known CVEs

Application Server:

- **PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1**

- ⚠ **CRITICAL FINDING:** PHP 5.6 reached End-of-Life (EOL) on December 31, 2018
- Approximately **8 years** without security updates
- Known vulnerable to numerous unpatched CVEs

Database:

- **MySQL** (Inferred from error messages)
 - Error message revealed: "mysql_connect(): Connection refused"

Frontend:

- HTML 4.01 Transitional (Legacy standard)
- JavaScript (inline)
- CSS (external stylesheet)
- Flash objects (deprecated technology)

Endpoint Categories Discovered

1. Static Resources (4 endpoints)

```
GET /images/logo.gif      → 200 OK
GET /images/remark.gif    → 200 OK
GET /style.css             → 200 OK
GET /favicon.ico          → 200 OK
```

2. Public Navigation Pages (4 endpoints)

```
GET /                      → 200 OK (index.php)
GET /categories.php        → 200 OK (browsed multiple times)
GET /artists.php          → 200 OK (browsed multiple times)
GET /disclaimer.php        → 200 OK
```

3. Dynamic Content with Parameters (2 endpoint patterns)

```
GET /artists.php?artist=1  → 200 OK
GET /listproducts.php?cat=X → Referenced in HTML (not directly requested)
GET /listproducts.php?artist=X → Referenced in HTML (not directly requested)
```

4. Authentication & Session Management (4 endpoints)

```
GET /login.php             → 200 OK (login form)
POST /userinfo.php         → 302 Found (failed authentication)
POST /userinfo.php         → 200 OK (successful auth, but DB error)
GET /userinfo.php          → 302 → login.php (requires authentication)
```

```
GET /cart.php          → 200 OK ("You are not logged on" error page)
GET /signup.php        → Referenced (signup disabled)
```

5. User Input Functionality (3 endpoints)

```
GET /guestbook.php      → 200 OK (form displayed)
POST /guestbook.php     → 200 OK (message submitted: "dededede")
POST /search.php?test=query → 200 OK (search: "erererer")
GET /comment.php?aid=1  → 404 Not Found (missing file)
```

6. AJAX/Advanced Features (1 endpoint)

```
GET /AJAX/index.php    → Referenced in navigation (not directly tested)
```

Parameters Identified

GET Parameters (4 unique):

Parameter	Purpose	Example Value	Potential Risk
cat	Category ID	1, 2, 3, 4	SQL Injection
artist	Artist ID	1, 2, 3	SQL Injection
aid	Artist/Comment ID	1	SQL Injection
test	Query string	query	Unknown purpose

POST Parameters (7 unique):

Parameter	Purpose	Example Value	Context
uname	Username	test	Authentication
pass	Password	test, testsd	Authentication
searchFor	Search query	erererer	Search functionality
goButton	Submit button	go	Search form
name	User name	anonymous user	Guestbook
text	Message text	dededede	Guestbook
submit	Submit button	add message	Guestbook form

Response Analysis

Successful Responses (200 OK):

- All navigation pages loaded successfully
- Forms rendered properly
- Static resources served without issues

Redirects (302 Found):

- `/userinfo.php` → `/login.php` (authentication required)
- Failed login attempts redirect to login page

Client Errors (404 Not Found):

- `/comment.php?aid=1` → File does not exist (potential information disclosure)

Database Errors:

- Login attempt with valid credentials (`test/test`) revealed:

```
Warning: mysql_connect(): Connection refused in
/hj/var/www/database_connect.php on line 2
Website is out of order. Please visit back later. Thank you for
understanding.
```

- ⚠ **CRITICAL:** Full server path disclosed: `/hj/var/www/database_connect.php`
- ⚠ **CRITICAL:** Deprecated `mysql_connect()` function (removed in PHP 7.0)
- MySQL connection failure indicates potential misconfiguration

Form Analysis

Forms Detected:

1. **Search Form** (Present on all pages)
 - Action: `search.php?test=query`
 - Method: POST
 - Fields: `searchFor`, `goButton`
2. **Login Form** (`/login.php`)
 - Action: `userinfo.php`
 - Method: POST
 - Fields: `uname`, `pass`
 - Test credentials revealed in UI: `test / test`
3. **Guestbook Form** (`/guestbook.php`)
 - Action: (self-posting)
 - Method: POST
 - Fields: `name` (hidden), `text`, `submit`
 - Default name: "anonymous user"

Session Management

Cookie Analysis:

- No session cookies observed in successful traffic to testphp.vulnweb.com
 - Authentication state managed server-side
 - Cart functionality requires active session
-

Step 3: Pentesting Strategy Recommendations

Based on the observed traffic patterns and application behavior, the following vulnerability categories are recommended for testing:

Priority 1: CRITICAL - SQL Injection

Rationale:

- Multiple GET parameters accept numeric IDs (`cat`, `artist`, `aid`)
- Outdated PHP version using deprecated MySQL functions
- No visible input validation or parameterized queries
- Search functionality processes user input

Test Locations:

```
GET /artists.php?artist=1
GET /listproducts.php?cat=1
GET /comment.php?aid=1
POST /search.php (searchFor parameter)
```

Recommended Payloads:

- Boolean-based: ' OR '1'='1
- Union-based: ' UNION SELECT NULL--
- Error-based: ' AND 1=CONVERT(int, @@version)--
- Time-based: '; WAITFOR DELAY '00:00:05'--

Testing Strategy:

1. Start with simple quote injection: `?artist=1'`
 2. Confirm error-based injection
 3. Enumerate database structure with UNION
 4. Extract data from identified tables
-

Priority 1: CRITICAL - Cross-Site Scripting (XSS)

Rationale:

- Guestbook accepts user input and displays it back

- Search functionality reflects user input in response
- No visible output encoding in HTML source
- Outdated PHP without built-in XSS protections

Test Locations:

```
POST /guestbook.php (text parameter)
POST /search.php (searchFor parameter)
GET /artists.php?artist=<payload>
```

XSS Payload Examples:

- Basic: `<script>alert('XSS')</script>`
- IMG tag: `<img src=x onerror=alert('XSS')>`
- Event handler: `<body onload=alert('XSS')>`
- Stored XSS: Test guestbook persistence

Testing Strategy:

1. Submit basic XSS payloads to guestbook
2. Verify if script executes on page reload (Stored XSS)
3. Test reflected XSS in search and parameters
4. Test XSS in error messages

Priority 1: HIGH - Authentication Bypass**Rationale:**

- SQL injection potential in login form
- Weak credential handling observed
- Test credentials hardcoded in UI
- Legacy authentication patterns

Test Vectors:

```
Username: admin' OR '1'='1'--
Password: anything
```

```
Username: admin'--
Password: (empty)
```

```
Username: ' OR 1=1--
Password: ' OR 1=1--
```

Testing Strategy:

1. Attempt SQL injection in username/password fields

2. Test for authentication bypass using SQL comments
 3. Brute force testing (if rate limiting absent)
 4. Test session fixation and hijacking
-

Priority 2: HIGH - Path Traversal & Local File Inclusion (LFI)

Rationale:

- Server path disclosed in error message (`/hj/var/www/`)
- PHP application with file operations likely
- 404 error on `comment.php` suggests file-based routing

Test Locations:

```
GET /artists.php?artist=../../../../../../../../etc/passwd
GET /listproducts.php?cat=../../../../../../../../etc/passwd
```

LFI Payload Examples:

- Basic: `../../../../../../../../etc/passwd`
 - Null byte (PHP < 5.3): `../../../../../../../../etc/passwd%00`
 - Wrapper: `php://filter/convert.base64-encode/resource=index.php`
-

Priority 2: HIGH - Information Disclosure

Findings Already Discovered:

- Database connection failure message shows full file path
- PHP version and nginx version exposed in headers
- 404 error reveals missing file (`comment.php`)
- Test credentials displayed in application

Additional Testing:

- Request non-existent pages to trigger error messages
 - Check for backup files: `.php.bak`, `.php~`, `.php.old`
 - Directory listing: `/images/`, `/AJAX/`, etc.
 - Check for exposed configuration files: `config.php`, `database.php`
-

Priority 2: MEDIUM - Cross-Site Request Forgery (CSRF)

Rationale:

- No CSRF tokens visible in forms
- State-changing operations without protection
- POST operations for guestbook and authentication

Test Locations:

- Guestbook submission
- Login form
- Cart operations (if accessible)

Proof of Concept:

```
<form action="http://testphp.vulnweb.com/guestbook.php" method="POST">
  <input name="name" value="anonymous user">
  <input name="text" value="CSRF Attack">
  <input name="submit" value="add message">
</form>
<script>document.forms[0].submit();</script>
```

Priority 3: MEDIUM - Insecure Direct Object References (IDOR)**Rationale:**

- Sequential IDs observed: `artist=1`, `artist=2`, `artist=3`
- No apparent access control on artist/category viewing
- Cart requires authentication (good)

Test Strategy:

1. Enumerate all artist IDs: `/artists.php?artist=1` through `?artist=100`
2. Enumerate all category IDs: `/listproducts.php?cat=1` through `?cat=100`
3. Test if negative IDs work: `?artist=-1`
4. Test if special characters work: `?artist=1a`

Priority 3: MEDIUM - Server-Side Request Forgery (SSRF)**Rationale:**

- Search functionality may fetch external resources
- AJAX endpoint may make backend requests

Test Locations:

- Search parameter
- Any URL/file upload functionality (if discovered)

Priority 3: LOW - Clickjacking**Rationale:**

- No X-Frame-Options header detected (though Google has it)
- Legacy application may lack modern security headers

Test:

```
<iframe src="http://testphp.vulnweb.com/login.php"></iframe>
```

Priority 3: LOW - Security Headers

Missing Security Headers:

- Content-Security-Policy (CSP)
- X-Content-Type-Options
- Referrer-Policy
- Permissions-Policy

Testing:

- Review all response headers for security hardening

Step 4: Effort Estimation

Based on the current in-scope traffic volume and application complexity, here is a breakdown of estimated penetration testing time:

Time Breakdown (Medium Complexity Application)

Phase	Tasks	Estimated Time
1. Reconnaissance & Mapping	- Verify scope - Spider application - Map all endpoints - Identify all parameters - Document technology stack	2-3 hours
2. Manual Testing - Critical	- SQL Injection (all parameters) - XSS (reflected & stored) - Authentication bypass - Session management testing	4-6 hours
3. Manual Testing - High/Medium	- IDOR testing - CSRF testing - File inclusion (LFI/RFI) - Information disclosure - Access control flaws	3-4 hours
4. Manual Testing - Low	- Security headers review - Clickjacking - SSRF testing - Business logic flaws	1-2 hours

Phase	Tasks	Estimated Time
5. Exploitation & Proof of Concept	- Develop working exploits - Document attack chains - Take screenshots/evidence - Test impact and scope	2-3 hours
6. Validation & Cleanup	- Retest findings - Confirm reproducibility - Rate severity (CVSS) - Remove test data	1-2 hours
7. Reporting & Documentation	- Write technical findings - Create executive summary - Provide remediation guidance - Prepare presentation	3-4 hours

Total Estimated Time: 16-24 hours (2-3 working days)

Factors Affecting Timeline:

Could Reduce Time:

- Application is intentionally vulnerable (faster exploitation)
- Small attack surface (~15 unique endpoints)
- No anti-automation controls (faster testing)
- Publicly documented as test site (less legal/ethical delays)

Could Increase Time:

- If attempting to chain vulnerabilities for maximum impact
- If performing comprehensive business logic testing
- If full database enumeration required
- If client requests proof-of-concept exploits for all findings

Recommended Approach:

Phase 1 - Quick Wins (4-6 hours):

- Focus on SQL Injection and XSS
- Test authentication bypass
- Document information disclosure

Phase 2 - Comprehensive Testing (10-14 hours):

- Complete testing of all vulnerability categories
- Develop working exploits
- Full documentation

Phase 3 - Advanced Scenarios (Optional, +4-8 hours):

- Attack chain development
 - Post-exploitation scenarios
 - Lateral movement testing
-

Summary & Key Takeaways

Application Profile

- **Type:** E-commerce demo/training application
- **Purpose:** Intentionally vulnerable for security testing
- **Complexity:** Low-Medium (basic CRUD operations, simple authentication)
- **Technology:** Legacy stack (PHP 5.6, nginx 1.19, MySQL)

Initial Assessment

Security Posture: POOR ⚠️

The application exhibits multiple indicators of weak security:

1. X Outdated, unsupported software (PHP 5.6 EOL since 2018)
2. X Information disclosure (paths, errors, versions)
3. X Deprecated functions (`mysql_connect()`)
4. X No visible input validation or output encoding
5. X Missing security headers
6. X Test credentials hardcoded in UI

Expected Finding Severity Distribution:

- **Critical:** 2-4 findings (SQL Injection, XSS, Auth Bypass)
- **High:** 3-5 findings (LFI, Session issues, Information Disclosure)
- **Medium:** 4-6 findings (IDOR, CSRF, Missing headers)
- **Low:** 2-3 findings (Clickjacking, Minor misconfigurations)

Next Steps

1. **Obtain Written Authorization** before proceeding with active testing
 2. **Establish Rules of Engagement** (testing hours, out-of-scope items, emergency contacts)
 3. **Begin with Reconnaissance Phase** using passive techniques
 4. **Proceed with Manual Testing** following priority order
 5. **Document All Findings** with reproducible steps and evidence
 6. **Provide Remediation Guidance** for each vulnerability identified
-

Ethical & Legal Considerations

⚠️ **IMPORTANT:** This assessment is based solely on passive traffic analysis. No active exploitation has been performed.

Before proceeding with active penetration testing:

- ✓ Ensure written authorization from the application owner
 - ✓ Define clear scope and boundaries
 - ✓ Establish communication channels for issues
 - ✓ Document all testing activities for accountability
 - ✓ Use non-production environments when possible
 - ✓ Have a rollback plan for any state-changing operations
-

Report Generated: January 2, 2026

Analyst: Security Assessment Team

Status: Preliminary Analysis - Awaiting Authorization for Active Testing